用 virtual pointer 阐述 SystemVerilog/UVM 多态（OOP）

本文基于 C++ 里的 virtual ptr 和 vtable，详细阐述 SV 里的 OOP 原理，剖析哪些是静态变量（compile），哪些是动态变量（runtime），以及 handle 类型和 obj 对象类型的区别。

这也是 UVM obj 调用的重中之重——UVM 底层 code 传递/调用 obj 时默认使用了 base obj，比如 driver 的 uvm_sequence_item 和 seq 里的 m_sequencer。

一、obj 三个核心组件：obj → vptr → vtable → vfunction

SV/UVM 用 handle 表达 class 指针、用 value 语义表达数组，因此 OOP 里的 inheritance 和 polymorphism 较难理解。掌握以下三层结构是关键：

组件	说明
① class 对象本体	包含普通成员变量（compile 静态分配）、动态成员变量（runtime 分配）、静态 function，以及指向 vtable 的 vptr
② virtual pointer（vptr）	编译器自动插入，每个含 virtual 的 class 对象 1 个；运行时用于动态派发，指向该对象所属 class 的 vtable
③ virtual table（vtable）	基于函数指针数组的地址表；存在于只读段，每个 class 一份（非每个 obj）；只收录 virtual 函数

二、vptr 实例：class Dog extends Animal

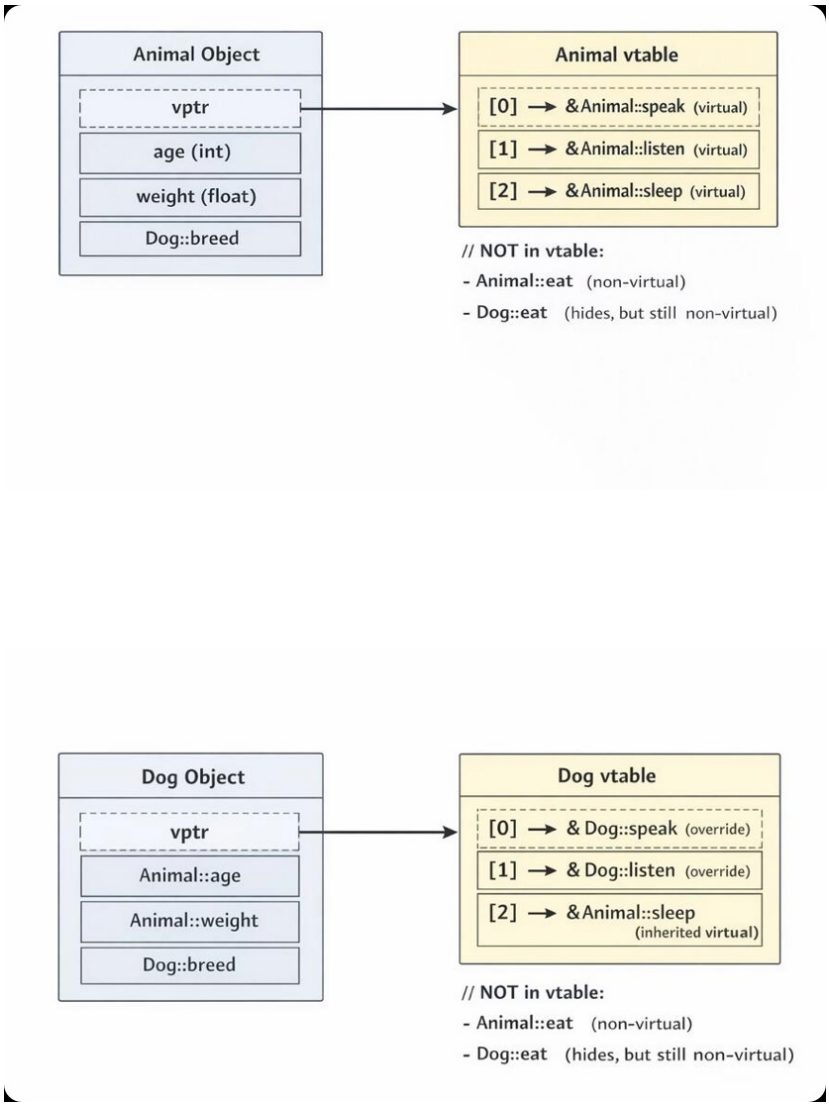
以下面的类定义为基础，贯穿全文所有例子：

```
class Animal;
  function    void eat();   endfunction  // NON-virtual
  virtual function void speak(); endfunction  // virtual
  virtual function void sleep(); endfunction  // virtual，Dog 不 override
endclass

class Dog extends Animal;
  function    void eat();       endfunction  // hides，非 override
```

```
 virtual function void speak();    endfunction  // override
 function    void only_in_dog(); endfunction  // Dog 独有
endclass
```

Animal / Dog 对象内存布局与 vtable 示意图：



图：Animal Object 和 Dog Object 各自的 vptr 指向对应 vtable；non-virtual 的 eat() 不在 vtable 中

vtable 内容对比：

函数	Animal vtable	Dog vtable	备注
speak()	&Animal::speak	&Dog::speak	Dog override，vtable[0] 指向新实现
listen()	&Animal::listen	&Dog::listen	Dog override，vtable[1] 指向新实现
sleep()	&Animal::sleep	&Animal::sleep	Dog 无 override，继承父类实现

函数	Animal vtable	Dog vtable	备注
eat()	不在 vtable	不在 vtable	non-virtual，不参与动态派发
only_in_dog()	不在 vtable	不在 vtable	non-virtual，Dog 独有

三、绑定规则与 function 调用结果

两条核心规则（与 C++ 完全一致）：

★ non-virtual → 看 handle 的静态类型（SV 里：声明 handle 时的类型）

★ virtual → 看 new 出来的真实对象类型（SV 里：运行时 vptr 所指的 class）

完整示例代码：

```
initial begin
  Animal a;
  Dog    d;
  d = new(age: 3, weight: 12.5, breed: 7);

  a = d;  // 向上转型：Animal handle 指向 Dog 对象

  // 通过 Animal handle a 调用
  a.speak();  // virtual  -> Dog::speak    看对象类型
  a.sleep();  // virtual  -> Animal::sleep   Dog 没 override
  a.eat();    // NON-virt -> Animal::eat    看 handle 静态类型

  // 通过 Dog handle d 调用
  d.speak();  // Dog::speak
  d.sleep();  // Animal::sleep
  d.eat();    // Dog::eat  静态类型是 Dog
end
```

四、SystemVerilog 调用方式 / 实际调用大表

第一部分：non-virtual（静态绑定，编译期决定）

调用方式	实际调用	原因
Dog d = new(); d.eat();	Dog::eat	静态类型 = Dog
Animal a = d; a.eat();	Animal::eat	non-virtual → 看 handle 静态类型
Animal a; a = new Dog(); a.eat();	Animal::eat	同上
$cast(Dog dd, a); dd.eat();	Dog::eat	handle 已是 Dog 类型
$cast(Dog dd, a); a.eat();	Animal::eat	仍用原 base handle a 调用

第二部分：virtual（动态派发，运行期决定）

调用方式	实际调用	原因
Dog d = new(); d.speak();	Dog::speak	override
Animal a = d; a.speak();	Dog::speak	virtual → 看对象真实类型
Animal a = new Animal(); a.speak();	Animal::speak	对象是 Animal
$cast(Dog dd, a); dd.speak();	Dog::speak	cast 成功，dd 是 Dog handle
$cast(Dog dd, a); a.speak();	Dog::speak	handle 不影响 virtual dispatch

第三部分：$cast 有效场景

调用方式	实际行为	说明
Animal a = d; Dog dd; $cast(dd, a); dd.only_in_dog();	成功调用	访问派生类独有 API，必须先 cast
$cast(dd, a); dd.eat();	Dog::eat	handle 已是 Dog，non-virtual 走 Dog 实现
$cast(dd, a); dd.speak();	Dog::speak	override 生效

第四部分：$cast 也"没用"的场景

场景 A：cast 后仍用原 base handle 调用

调用方式	实际调用	原因
$cast(Dog dd, a); a.eat();	Animal::eat	换了 dd 不等于换了 a，a 仍是 Animal handle
$cast(Dog dd, a); a.speak();	取决于对象类型	virtual 不看 handle，看 vptr 指向的对象

场景 B：对象根本不是 Dog（cast 失败）

调用方式	实际行为
Animal a = new Animal();	对象真实类型是 Animal
$cast(Dog dd, a);	运行期错误：cast 失败，dd = null
dd.eat();	运行期错误：null handle 非法访问

正确做法：

```
Animal a;
Dog real_dog = new();    // 先 new 一个真正的 Dog
a = real_dog;            // 向上转型，a 指向 Dog 对象
Dog dd;
if ($cast(dd, a)) begin   // cast 成功，因为 a 确实指向 Dog
  dd.only_in_dog();
end
```

第五部分：base handle 找不到函数（编译期错误）

调用方式	结果	原因
Animal a = d; a.only_in_dog();	编译错误	Animal 类没有 only_in_dog()
Animal a = d; a.breed = 3;	编译错误	Animal 类没有 breed 字段
Dog dd; $cast(dd, a); dd.only_in_dog();	正确	先 cast 到 Dog handle 再调用

第六部分：最容易误解的组合场景

调用方式	实际调用	易错点
Animal a = d; a.eat();	Animal::eat	误以为会分派到 Dog::eat
Animal a = d; a.speak();	Dog::speak	virtual 正确分派，这个是对的
$cast(Dog dd, a); dd.eat();	Dog::eat	误以为是 cast 改变了结果；实际是 handle 类型变了
$cast(Dog dd, a); a.eat();	Animal::eat	误以为 cast 影响了 a；a 没变
$cast(Dog dd, a); dd.only_in_dog();	成功	需要 cast 才能访问派生类 API
Animal a = new(); $cast(Dog dd, a);	失败（runtime）	真实对象是 Animal，不是 Dog

五、核心规则总结

规则	说明
non-virtual 绑定	编译期决定，看 handle 的静态类型（声明类型）
virtual dispatch	运行期决定，看 new 出来的真实对象类型（vptr → vtable）
$cast 的作用	改变"你用来调用的 handle 类型"，从而访问派生类专有 API；不会改变 non-virtual 的绑定规则
编译期 vs 运行期	找不到函数 → 编译期；non-virtual 绑定 → 编译期；virtual dispatch → 运行期；$cast 成功/失败 → 运行期

六、UVM 现实意义映射表

如果你做 UVM，以上规则直接对应以下常见场景：

UVM 现象	背后原因 / 对应规则
driver 里 $cast(req, base_req)	恢复真实类型（base handle 没有派生字段）；避免编译错误或向错误 ref 赋值
scoreboard 按真实 item 类型分流	virtual + $cast：先用 virtual 函数识别类型，再 cast 到具体类型处理
req.some_field 编译错误	base handle 不含派生字段，需先 $cast 到具体类型
uvm_object c = req.clone(); DogItem di; $cast(di, c);	clone() 返回类型是 uvm_object（基类）；需 $cast 恢复真实类型
uvm_declare_p_sequencer	$cast(p_sequencer, m_sequencer) 防止跑错 sequencer 类型

来源：小红书 @Yang